

AND 9/20/26

Purpose:

Use the PIC16F883 interrupt system to observe how asynchronous hardware events temporarily change program execution, how multiple interrupt sources share one interrupt vector, and why interrupt service routines must preserve processor context

The lab begins with one interrupt source and an observable main program, adds a second source, expands PORTB interrupt-on-change to multiple pins, and then recreates the inherited priority-interrupt behavior with deliberately nested interrupt execution. Mastery reproduces the same visible behavior without nested interrupts or long delays inside an ISR

You should be able to explain what the processor was doing before an interrupt, what caused the execution to change, how the source was identified and serviced, what context must be preserved, and how execution safely resumes

Standards & references:

- RCET 3375 Lab Standard
- RCET PIC-AS Style Guide
- RCET Flowchart Guide
- PIC16F883 Datasheet, especially:
 - Section 2.3.2 - Stack
 - Section 3.4.3 - Interrupt-on-Change
 - Section 14.3 - Interrupt
 - Section 14.4 & Example 14-1 - Context saving During Interrupts
- PICmicro Mid-Range MCU Family Reference Manual, section 8 Interrupts
- Previous RCET 3375 lab book documentation

See rascimo Lab 05 ISRs Github for all links

Equipment & materials:

- MPLABXIDE & pic-as toolchain
- logic Analyzer (optional)
- 4MHz crystal oscillator circuit
- breadboard, jumpers, & interface components as required
- PICkit programmer/debugger
- LEDs & current-limiting resistors
- DLG7137 Display
- PIC16F883 circuit
- 4-Channel oscilloscope
- Lab book

AND 9/20/26

Part 1 - External Interrupt & Interrupted Main Execution:

Goal: Configure RB0/INT for a falling-edge interrupt. Each valid interrupt increments PORTC while main continuously toggles PORTA monitoring pin. Use a single-sequence oscilloscope capture to observe the interrupt request, interruption of main-line execution, and the resulting count change.

Required Behavior:

- Configure RB0/INT for a falling-edge external interrupt
- Choose any suitable unused PORTA digital output as the main-loop monitoring pin. Document the choice
- Main toggles the selected PORTA pin continuously and as quickly as the program structure reasonably allows
- Each external interrupt increments PORTC by one
- Display PORTC on LEDs as a binary count
- Do not poll RB0/INT from main
- Use the W/STATUS save/restore sequence in PIC16F883 datasheet section 14.4, Example 14.1 as the starting point for processor context handling

Before lab:

Prepare or reference:

- Pushbutton, PORTC LED, & PORTA monitoring-point schematic
- loading/electrical-limit analysis
- main/interrupt flowchart
- new SFR maps & relevant bits
- source code
- port-pin assignment map
- predicted oscilloscope behavior
- external-interrupt setup sequence
- context save/restore plan based on datasheet Example 14-1

AND 9/21/26

Lab 5 Interrupt Service Routines

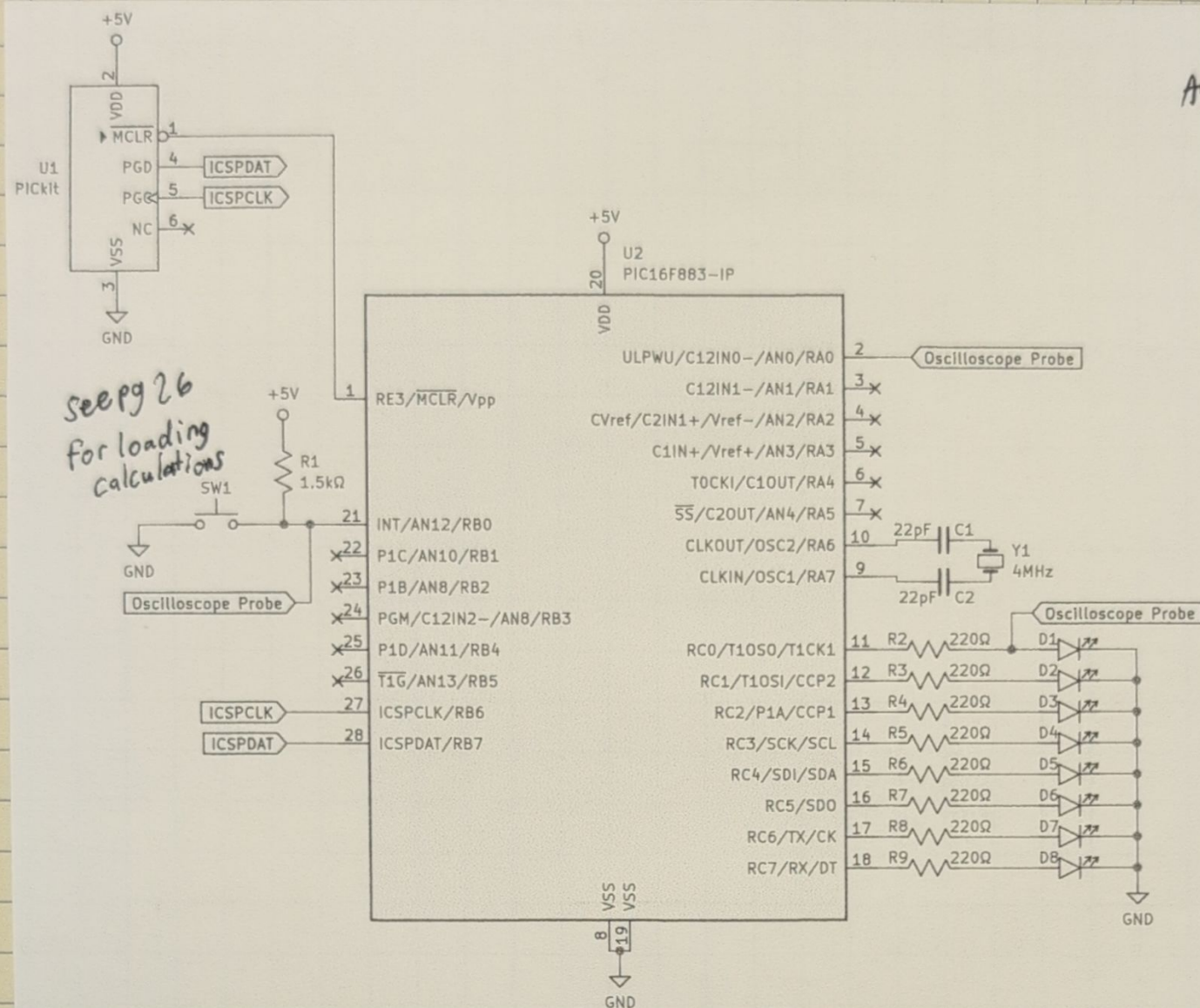


Figure 5.1 full schematic for PORT A monitoring circuit

REGISTER 2-3: INTCON: INTERRUPT CONTROL REGISTER

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-x
GIE	PEIE	TOIE	INTE	RBIE ⁽¹⁾	TOIF ⁽²⁾	INTF	RBIF
bit 7							bit 0

Legend:

R = Readable bit
-n = Value at POR

W = Writable bit
'1' = Bit is set

U = Unimplemented bit, read as '0'
'0' = Bit is cleared

x = Bit is unknown

bit 7	GIE: Global Interrupt Enable bit 1 = Enables all unmasked interrupts 0 = Disables all interrupts
bit 6	PEIE: Peripheral Interrupt Enable bit 1 = Enables all unmasked peripheral interrupts 0 = Disables all peripheral interrupts
bit 5	TOIE: Timer0 Overflow Interrupt Enable bit 1 = Enables the Timer0 interrupt 0 = Disables the Timer0 interrupt
bit 4	INTE: INT External Interrupt Enable bit 1 = Enables the INT external interrupt 0 = Disables the INT external interrupt
bit 3	RBIE: PORTB Change Interrupt Enable bit ⁽¹⁾ 1 = Enables the PORTB change interrupt 0 = Disables the PORTB change interrupt
bit 2	TOIF: Timer0 Overflow Interrupt Flag bit ⁽²⁾ 1 = TMR0 register has overflowed (must be cleared in software) 0 = TMR0 register did not overflow
bit 1	INTF: INT External Interrupt Flag bit 1 = The INT external interrupt occurred (must be cleared in software) 0 = The INT external interrupt did not occur
bit 0	RBIF: PORTB Change Interrupt Flag bit 1 = When at least one of the PORTB general purpose I/O pins changed state (must be cleared in software) 0 = None of the PORTB general purpose I/O pins have changed state

Note 1: IOCB register must also be enabled.

Note 2: TOIF bit is set when Timer0 rolls over. Timer0 is unchanged on Reset and should be initialized before clearing TOIF bit.

Figure 5.2 INTCON Register Used for Interrupts

AND 9/21/26

Lab 5 Interrupt Service Routines

65

AND 9/21/26

REGISTER 2-2: OPTION_REG: OPTION REGISTER							
R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
RBPV	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

Legend:
R = Readable bit W = Writable bit U = Unimplemented bit, read as '0'
-n = Value at POR '1' = Bit is set '0' = Bit is cleared x = Bit is unknown

bit 7 **RBPV**: PORTB Pull-up Enable bit
1 = PORTB pull-ups are disabled
0 = PORTB pull-ups are enabled by individual PORT latch values

bit 6 **INTEDG**: Interrupt Edge Select bit
1 = Interrupt on rising edge of INT pin
0 = Interrupt on falling edge of INT pin

bit 5 **T0CS**: Timer0 Clock Source Select bit
1 = Transition on T0CKI pin
0 = Internal instruction cycle clock (Fosc/4)

bit 4 **T0SE**: Timer0 Source Edge Select bit
1 = Increment on high-to-low transition on T0CKI pin
0 = Increment on low-to-high transition on T0CKI pin

bit 3 **PSA**: Prescaler Assignment bit
1 = Prescaler is assigned to the WDT
0 = Prescaler is assigned to the Timer0 module

bit 2-0 **PS<2:0>**: Prescaler Rate Select bits

Bit Value	Timer0 Rate	WDT Rate
000	1:2	1:1
001	1:4	1:2
010	1:8	1:4
011	1:16	1:8
100	1:32	1:16
101	1:64	1:32
110	1:128	1:64
111	1:256	1:128

Figure 5.3 OPTION_REG register Used for Interrupt Edge select bit

```

MOVWF W_TEMP      ;Copy W to TEMP register
SWAPF STATUS,W    ;Swap status to be saved into W
                   ;Swaps are used because they do not affect the status bits
MOVWF STATUS_TEMP ;Save status to bank zero STATUS_TEMP register
:
: (ISR)            ;Insert user code here
:
SWAPF STATUS_TEMP,W ;Swap STATUS_TEMP register into W
                   ;(sets bank to original state)
MOVWF STATUS      ;Move W into STATUS register
SWAPF W_TEMP,F    ;Swap W_TEMP
SWAPF W_TEMP,W    ;Swap W_TEMP into W

```

Figure 5.4 Example ISR Handler code

```

ISRHandler:
    MOVWF W_TEMP ;Copy W to TEMP register
    SWAPF STATUS,W ;Swap status to be saved into W
    ;Swaps are used because they do not affect the status bits
    MOVWF STATUS_TEMP ;Save status to bank zero STATUS_TEMP register

    INCF PORTC ;Increment PORTC
    BCF INTCON,1 ;Clear the flag on INT
    goto LeaveISR

LeaveISR:
    SWAPF STATUS_TEMP,W ;Swap STATUS_TEMP register into W
    ;(sets bank to original state)
    MOVWF STATUS ;Move W into STATUS register
    SWAPF W_TEMP,F ;Swap W_TEMP
    SWAPF W_TEMP,W ;Swap W_TEMP into W
    RETFIE

End

```

Figure 5.5 Used ISR Aandler code

AND 9/21/26

AND 9/21/26

Setup:

```

; add device setup here
; Bank 0
BCF 0x03,6 ;Set Bank 0
BCF 0x03,5 ;"
CLRF PORTA ;Clearing PORTA on reset
CLRF PORTB ;Clearing PORTB on reset
CLRF PORTC ;Clearing PORTC on reset

; Bank 1
BCF 0x03,6 ;Set Bank 1
BSF 0x03,5 ;"
CLRF TRISA ;Making PORTA an output
CLRF TRISC ;Making PORTC an output

; Bank 3
BSF 0x03,6 ;Set Bank 3
BSF 0x03,5 ;"
CLRF ANSEL ;Making PORTA digital I/O
CLRF ANSELH ;Making PORTB digital I/O

;Interrupts
BCF 0x03,6 ;Set Bank 1
BSF 0x03,5 ;"
BCF OPTION_REG,6;Allows for interrupt on falling edge
BCF INTCON,1
BSF INTCON,4 ;Enables the INT external Interrupt bit
BSF INTCON,7 ;Enables global interrupts

BCF 0x03,6 ;Set Bank 0
BCF 0x03,5 ;"
goto Main

```

Main:

```

; main code goes here
MOVLW 0x01 ;1 TCY
XORWF PORTA,1 ;1 TCY
goto Main

```

Figure 5.6 Setup & Main Code for ISR

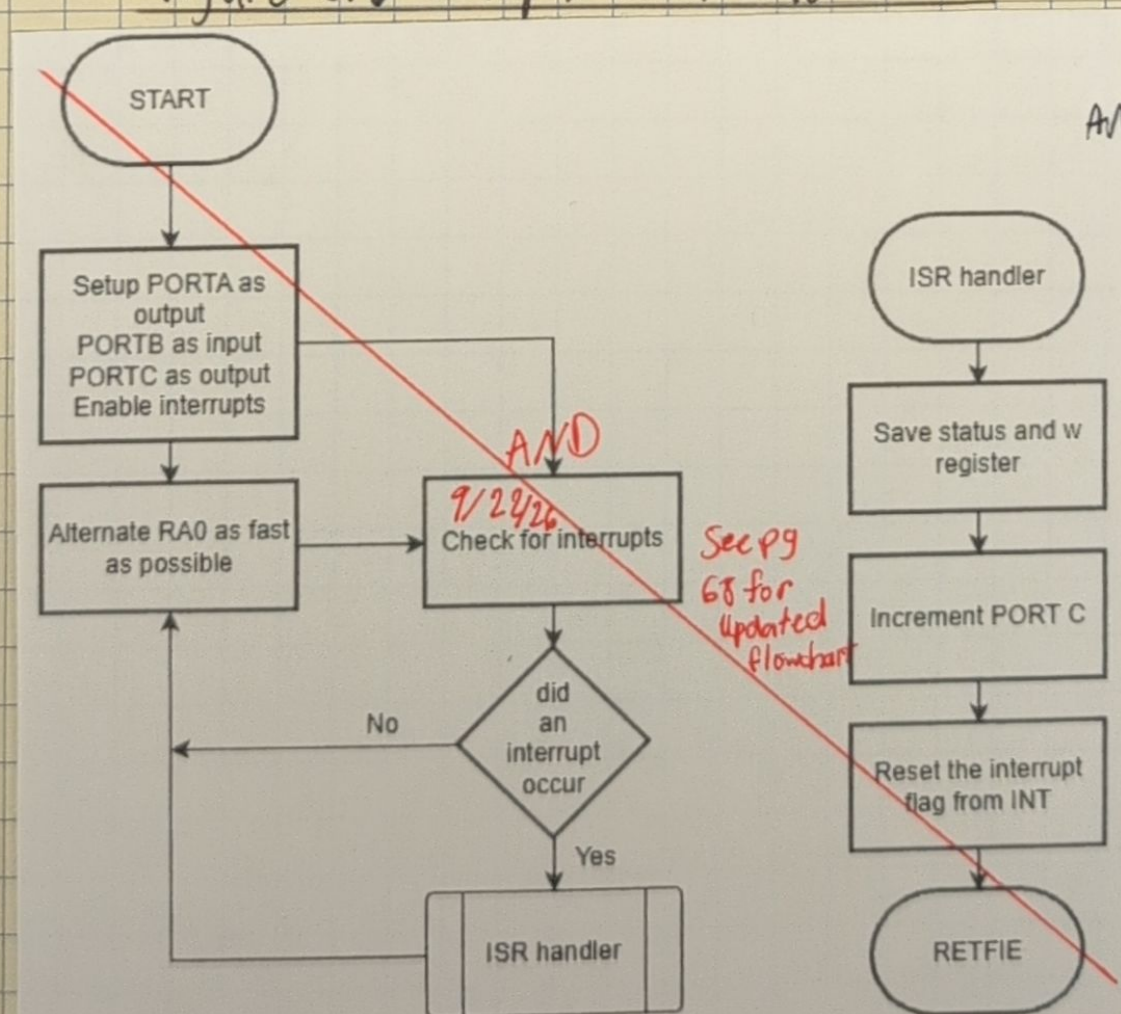


Figure 5.7 Flowchart for ISR handler code

AND 9/24/26

In the Lab:

1. Verify the PORTA monitoring signal
2. Verify the initial PORTC count
3. Press the RB0/INT button & confirm that the PORTC increments
4. Observe whether switch bounce creates additional interrupt events
 - When pressing the button there were a few times where multiple interrupts did occur and increased the PORTC count
5. Configure a single-sequence oscilloscope acquisition triggered on the falling edge of RB0/INT
6. Capture:
 - CH1 RB0/INT button signal
 - CH2 PORTA main-loop monitoring pin
 - CH3 RC0, the count LSB

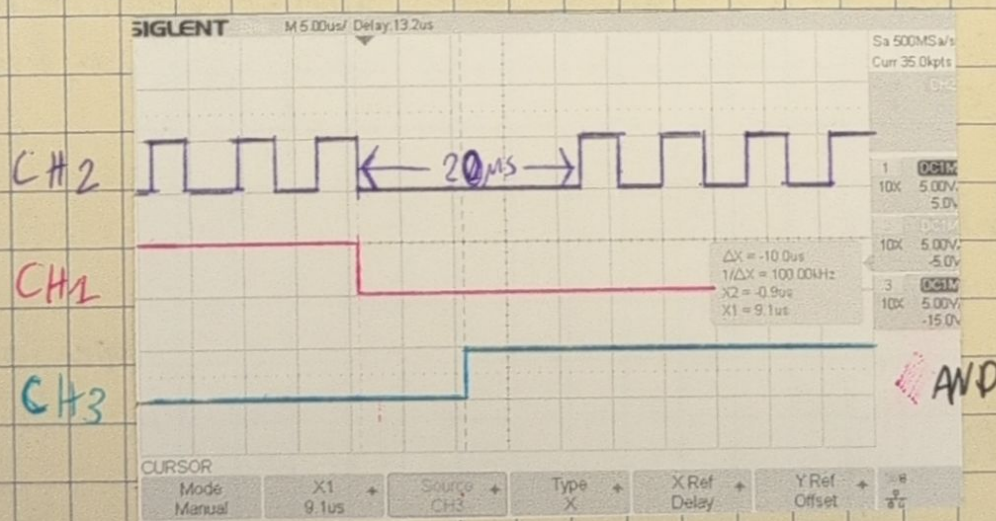


Figure 5.8 Capture of single button interrupt

7. Identify where main execution is interrupted & where the count changes
8. Save & explain the capture in the lab book

- After the interrupt button press the code runs through the interrupt service handler which takes approximately 20 μ s which was the amount of time it took before the count on PORTA restarted

TJR
9-22-26

AND 9/21/26

AND 9/22/26

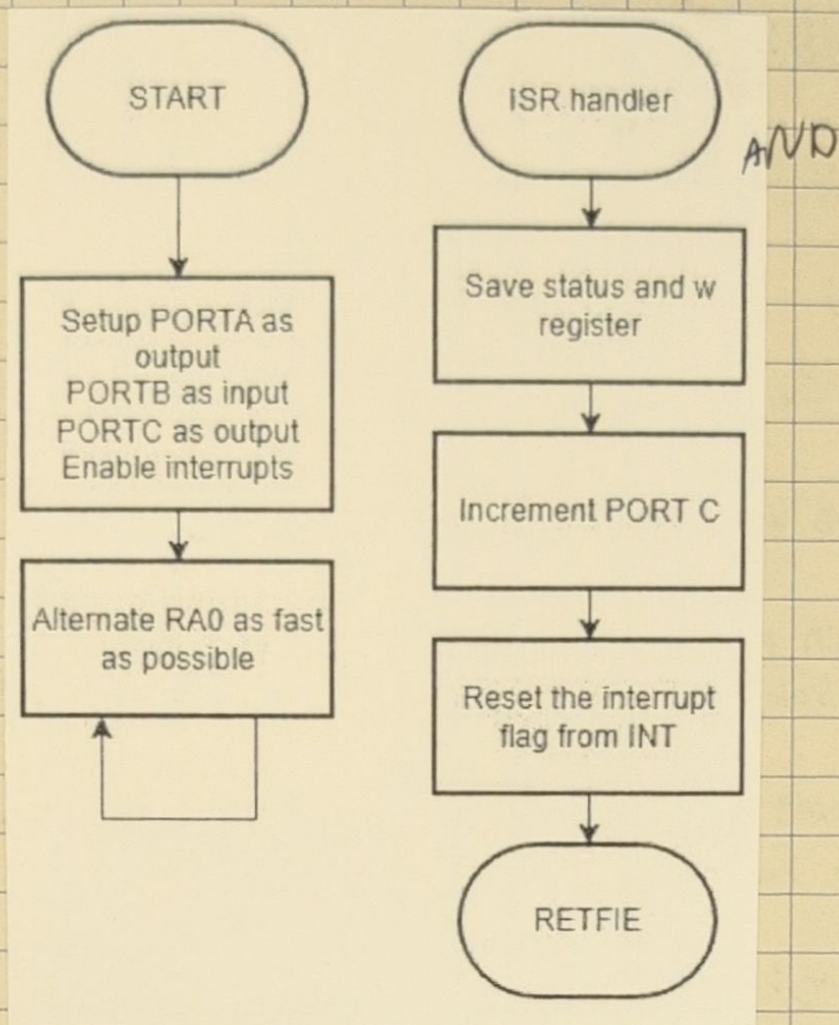


Figure 5.9 Updated Flowchart for ISR handler

Part 2 - Two Interrupt Sources & one Interrupt Vector :

Goal: Add RB4 interrupt-on-change while retaining RB0/INT. RB4 IOC increments PORTC; falling-edge RB0/INT clears PORTC. The emphasis is source determination, per-source service, & one common ISR exit path.

Required behavior:

- Retain the PORTA main loop monitoring signal chosen in Part 1
- Retain PORTC LEDs as the binary count
- Configure RB4 as the IOC input
- An RB4 change event increments PORTC
- A falling-edge RB0/INT event clears PORTC to zero
- Main polls neither source
- Both sources enter through the same interrupt vector

69

AND 9/22/26

AND 9/22/26

REGISTER 3-8: IOCB: INTERRUPT-ON-CHANGE PORTB REGISTER

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
IOCB7	IOCB6	IOCB5	IOCB4	IOCB3	IOCB2	IOCB1	IOCB0
bit 7							bit 0

Legend:

R = Readable bit

W = Writable bit

U = Unimplemented bit, read as '0'

-n = Value at POR

'1' = Bit is set

'0' = Bit is cleared

x = Bit is unknown

bit 7-0

IOCB<7:0>: Interrupt-on-Change PORTB Control bit

1 = Interrupt-on-change enabled

0 = Interrupt-on-change disabled

AND

Figure 5.10 IOCB Used to enable Interrupt on change.

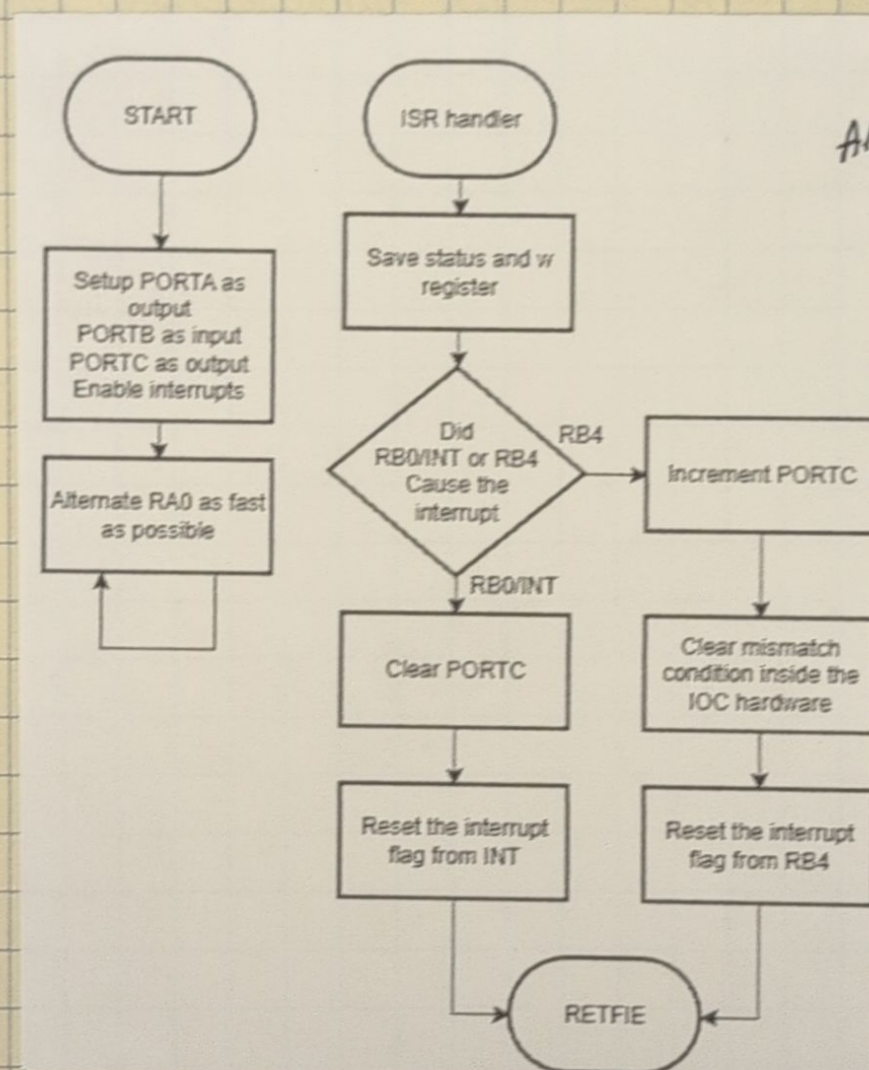


Figure 5.12 Updated flowchart for dual Interrupts

```

ISRHandler:
    MOVF W_TEMP, W    ;Copy W to TEMP register
    SWAPF STATUS, W    ;Swap status to be saved into W
    CLRF STATUS        ;Set to Bank 0
    ;Swaps are used because they do not affect the status bits
    MOVF STATUS_TEMP, W ;Save status to bank zero STATUS_TEMP register

    BTFSC INTC0N, 0    ;Tests for PORTB (RB4) interrupt flag
    goto IncrementPORTC
    BTFSC INTC0N, 1    ;Tests for INT interrupt flag
    goto ClearPORTC
    goto LeaveISR

IncrementPORTC:
    MOVF PORTC, W      ;clear mismatch condition
    INCF PORTC, F       ;Increment PORTC on every interrupt
    BCF INTC0N, 0
    goto LeaveISR

ClearPORTC:
    CLRF PORTC
    BCF INTC0N, 1
    goto LeaveISR

LeaveISR:
    SWAPF STATUS_TEMP, W ;Swap STATUS_TEMP register into W
    ;Sets bank to original state
    MOVF STATUS, W      ;Move W into STATUS register
    SWAPF W_TEMP, F     ;Swap W_TEMP
    SWAPF W_TEMP, W     ;Swap W_TEMP into W
    RETFIE
  
```

AND

AND 9/22/26

Figure 5.13 source code for Interrupts

In the Lab:

1. Verify that RB4 IOC increments PORTC
2. Generate several RB4 changes & confirm that the count accumulates
3. Trigger RB0/INT & confirm that PORTC clears
4. Test events occurring close together
5. Keep these four signals connected to the oscilloscope:
 - CH1 RB0/INT
 - CH2 Main-Loop monitoring pin
 - CH3 RCD
 - CH4 RB4 IOC Input
6. Capture an RB4 IOC event in single-sequence mode & explain the request, main-line interruption, & count change



Figure 5.14 Capture of RB4 Interrupt

When the interrupt on RB4 occurs it stops PORTA from toggling in main during the length of the interrupt which was about 23ms, then during the RB4 interrupt it increments PORTC by 1. Also before incrementing Port C, the IOL works by comparing PORTB's pins against a snapshot taken at the last read, but the pin change causes a mismatch so by reading PORTB it refreshes that snapshot to then compare it later if/when RB4 interrupt occurs again

2. Set the PORTC count so RCD is high, then capture an RB0/INT event in a single-sequence mode & explain the request, main-line interruption, & clear-to-zero result

AND 9/22/26

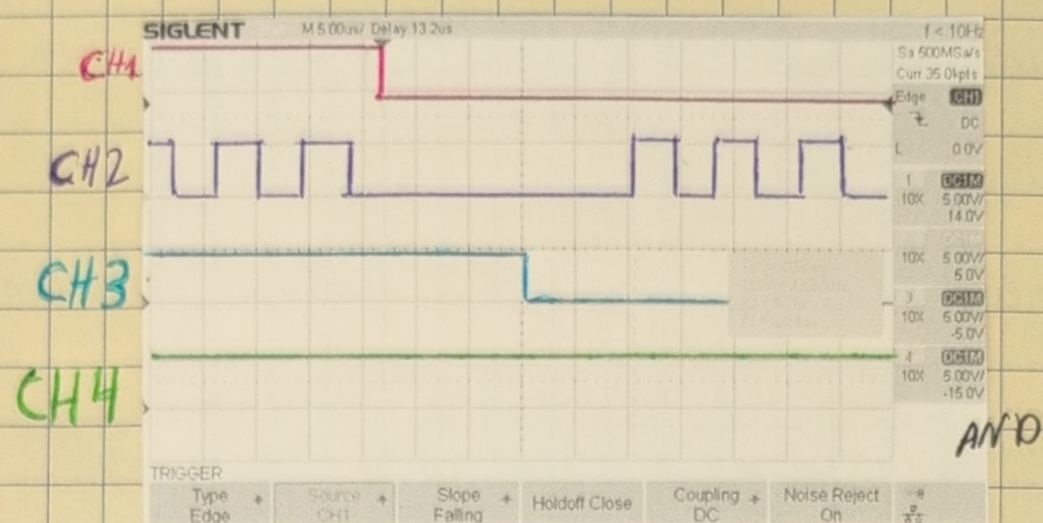


Figure 5.15 Capture of RBO/INT Interrupt

When the interrupt occurs it will stop PORTA from alternating then clears the count on PORTC, PORTA will then start alternating again after about 23 μ s which is the amount of time it takes to get through the ISR handler.

TJR
9-23-26

Part 3 - Determine Which PORTB Pin Changed:

Goal: Use interrupt-on-change on all eight PORTB pins & determine in software which pin changed. Display the detected pin number in binary on three PORTA LEDs

Required behavior:

- Disable the dedicated external interrupt for this part
- Enable IOC for RBO through RB7 using IOCB
- Establish the initial PORTB comparison state before enabling IOC
- Treat the IOC flag as notification that a PORTB change occurred, not as identification of a particular pin
- Determine which PORTB bit changed from the current and previous port states or an equivalent method
- Display the pin number on three PORTA outputs:
 - 000 = RBO - 001 = RB1 ... - 111 = RB7

AND 9/23/26

Required behavior cont.:

- Define a deterministic rule for the case where more than one changed bit is present when the ISR evaluates the event
- Correctly update the PORTB state used for comparison
- Correctly service & clear IOC before leaving the handler

Before Lab:

Prepare or reference:

- PORTB input & PORTA LED schematic
- IOC configuration documentation
- pin-assignment map
- source-determination algorithm
- flowchart
- deterministic multi-change rule
- source code
- register-style map for any named state/status storage used by the design
- the effect of the PICkit/ICSP connection on RB6 & RB7 while those pins are used as IOC inputs

[illegible]

Figure 5-16 PORTB Interrupt Selection Schematic

```

up:
; add device setup here
; Bank 0
BCF 0x02,6      ;Set Bank 0
BSF 0x02,5      ;"
CLRF PORTA      ;Clearing PORTA on reset
CLRF PORTE      ;Clearing PORTE on reset
CLRF PORTC      ;Clearing PORTC on reset

; Bank 1
BCF 0x02,6      ;Set Bank 1
BSF 0x02,5      ;"
CLRF TRISA      ;PORTA input
CLRF TRISC      ;PORTC input
MOVWF 0x0F      ;
MOVWF INISE      ;PORTE inputs
MOVWF KEPU      ;weak pull-ups on all pins
MOVWF IOCB      ;interrupt-on-change on RB0-RB7
BCF OPTION_REG,7;Pull-ups enabled

; Bank 2
BSF 0x02,6      ;Set Bank 3
BSF 0x02,5      ;"
CLRF ANSEL      ;Making PORTA digital I/O
CLRF ANSELH     ;Making PORTE digital I/O

;Interrupts
BCF 0x02,6      ;Set Bank 1
BSF 0x02,5      ;"
BCF OPTION_REG,6;allows for interrupt on falling edge
BCF INTCON,0
BSF INTCON,2     ;Enables the INT external Interrupt bit
BCF INTCON,4     ;INT disabled
BSF INTCON,7     ;Enables global interrupts

BCF 0x02,6      ;Set Bank 0
BCF 0x02,5      ;"
goto Main

```

Figure 5.17 Setup code for IC0B

When setting up the code I disabled the INT1 interrupt but enabled RBIF to enable interrupt on change for PORTB then IOCB also needed to be active to enable interrupt on change for PORTB

AND
9/23/26

AND 9/23/26

AND 9/24/26

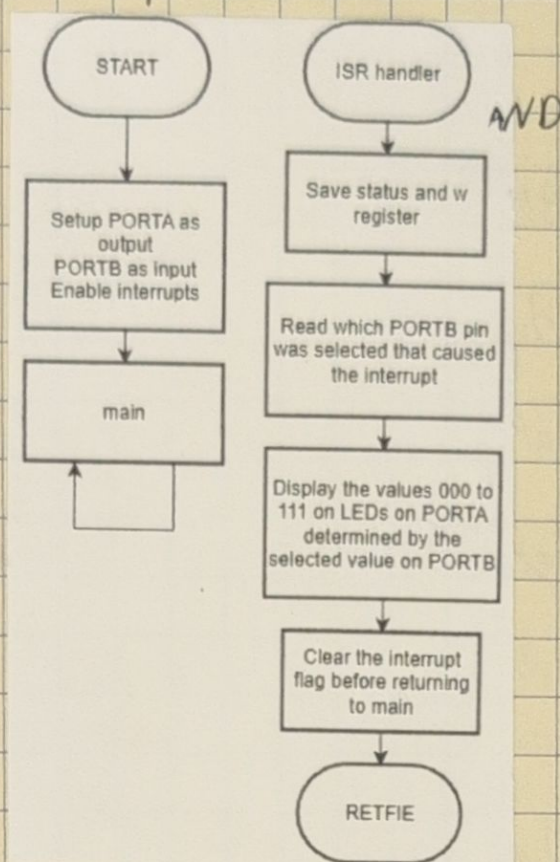


Figure 5.18 Flowchart for selection interrupts.

```

ISRHandler:
    MOVWF W_TEMP ;Copy W to TEMP register
    SWAPF STATUS,W ;Swap status to be saved into W
    ;Swaps are used because they do not affect the status bits
    MOVWF STATUS_TEMP ;Save status to bank zero STATUS_TEMP register

    MOVF PORTB,W ;Compare current PORTB vs. previous PORTB
    MOVWF PORTB_NEW
    XORWF PORTB_OLD,W
    MOVWF CHANGED

    MOVF PORTB_NEW,W ;Save current PORTB for next comparison
    MOVWF PORTB_OLD

    BTFSF CHANGED,0
    goto Display0
    BTFSF CHANGED,1
    goto Display1
    BTFSF CHANGED,2
    goto Display2
    BTFSF CHANGED,3
    goto Display3
    BTFSF CHANGED,4
    goto Display4
    BTFSF CHANGED,5
    goto Display5
    BTFSF CHANGED,6
    goto Display6
    BTFSF CHANGED,7
    goto Display7
    goto IOC_Flag

Display0:
    MOVWF 0x00 ;Display 000 on PORTA
    goto SetDisplay
Display1:
    MOVWF 0x01 ;Display 001 on PORTA
    goto SetDisplay
Display2:
    MOVWF 0x02 ;Display 010 on PORTA
    goto SetDisplay
Display3:
    MOVWF 0x03 ;Display 011 on PORTA
    goto SetDisplay
Display4:
    MOVWF 0x04 ;Display 100 on PORTA
    goto SetDisplay
Display5:
    MOVWF 0x05 ;Display 101 on PORTA
    goto SetDisplay
Display6:
    MOVWF 0x06 ;Display 110 on PORTA
    goto SetDisplay
Display7:
    MOVWF 0x07 ;Display 111 on PORTA

SetDisplay:
    MOVWF PORTA ;Display the selected value on PORTA

IOC_Flag:
    BCF INCON,0 ;Clear the REIF flag after PORTB was read

;Leave ISR:
    SWAPF STATUS_TEMP,W ;Swap STATUS_TEMP register into W
    ;sets bank to original state!
    MOVWF STATUS ;Move W into STATUS register
    SWAPF W_TEMP,F ;Swap W_TEMP
    SWAPF W_TEMP,W ;Swap W_TEMP into W
    RETFIE
    
```

Figure 5.19 ISR handler to Display values on PORTA

AND 9/24/26

AND 9/24/26

In the Lab:

1. Verify RB0 through RB7 individually
2. Confirm the correct three-bit pin number for every input
3. Test repeated changes on the same pin
4. Test different pins in sequence
5. Create at least one condition where more than one change may be present & verify the documented rule
6. Confirm that the ISR doesn't become stuck servicing an uncleared IOC condition
7. Record any effect of bounce or closely spaced input changes on source determination
8. Verify RB6 & RB7 with the PICkit/ICSP connection in the configuration you intend to use during normal operation

When some bounce occurred ~~since~~ since my code looked for what bit caused the interrupt the bounce will display what pin produced the bounce

Part 4 - Priority Interrupts, Nested Execution, & Stack Depth:

Goal: Create 1/6/7 priority-interrupt behavior using the DLG7137 display on PORTC from earlier I/O Labs

Required visible behavior:

Reuse the existing DLG7137/PORTC interface. Reference the earlier lab-book schematic & display encoding unless something changes

- The display on PORTC must precisely display 1, 6, or 7 at any time to reflect the current state of the system
- Main displays 1 continuously
- Falling-edge RB0/INT indicates the 6 Behavior

JP
1-24-26

AND 9/24/26

Required visible behavior cont.:

- IBC on RB1 indicates the 7 behavior
- 6 displays for two seconds
- 7 displays for two seconds
- 7 may interrupt 6 at any time while 6 is active
- While 7 is active, no lower-priority behavior may interrupt it
- When 7 finishes, execution returns to the state that was active when 7 began
 - If 7 interrupted 6, 6 resumes & completes its remaining time before returning to 1
 - If 7 interrupted the normal 1 state, the display returns to 1
- 7 may interrupt the same 6 behavior multiple times. Each time, 6 resumes its remaining time after 7 finishes

Delay & subroutine requirement:

Implement the long delays with callable subroutines using CALL/RETURN. Use nested delay/subroutine calls as appropriate. Normal subroutine calls are intentionally part of the stack-depth problem in this part.

Required stack analysis:

Before demonstration, determine the worst-case hardware stack depth of the final design. Account for every return address that can be active in the worst case including:

- Call depth before the first interrupt
- The 6 interrupt return address
- Calls made while executing the 6 behavior
- The nested 7 interrupt return address
- Calls made while executing the 7 behavior
- repeated 7 interruptions of 6 as allowed by the design

Level	Pushed by	Return address points into	Popped by
1 (bottom)	Hardware, RB0/INT fires in Main	Main loop	RETFIE in LeaveNormal
2	CALL Delay2Sec in Service6	Service6 (cleanup code after the call)	RETURN at end of Delay2Sec
3	CALL OutDelay in Delay2Sec (6's copy)	Delay2Sec (next iteration / final RETURN)	RETURN at end of OutDelay→MidDelay→InDelay chain
4	Hardware, RB1 IOC fires <i>inside</i> OutDelay/MidDelay/InDelay (GIE was re-armed by Service6)	wherever the delay loop was executing for 6	RETFIE in LeaveNested
5	CALL Delay2Sec in Service7	Service7 (resync code after the call)	RETURN at end of Delay2Sec
6 (top)	CALL OutDelay in Delay2Sec (7's copy)	Delay2Sec (next iteration / final RETURN)	RETURN at end of OutDelay→MidDelay→InDelay chain

Before Lab:

- DLG 7137/PORTC schematic
- RBO & RB1 interrupt & IOC wiring
- Interrupt/IOC documentation
- Pin assignment map
- nested-interrupt flowchart
- nested context-save strategy
- delay subroutine structure
- Worst-case hardware stack analysis
- Source code

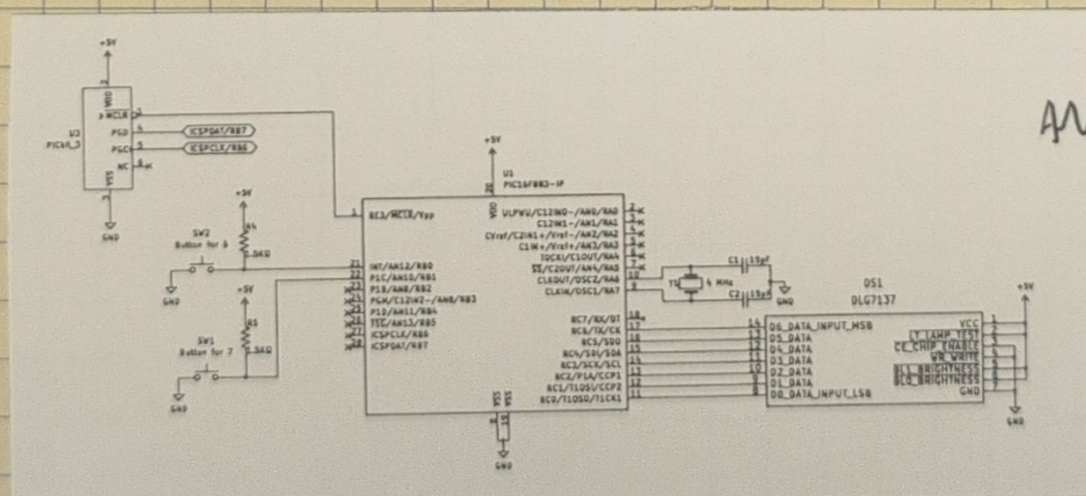


Figure 5.20 full schematic for DLG7137 Display

Lab 5 Interrupt Service Routines

79

AND 9/25/26

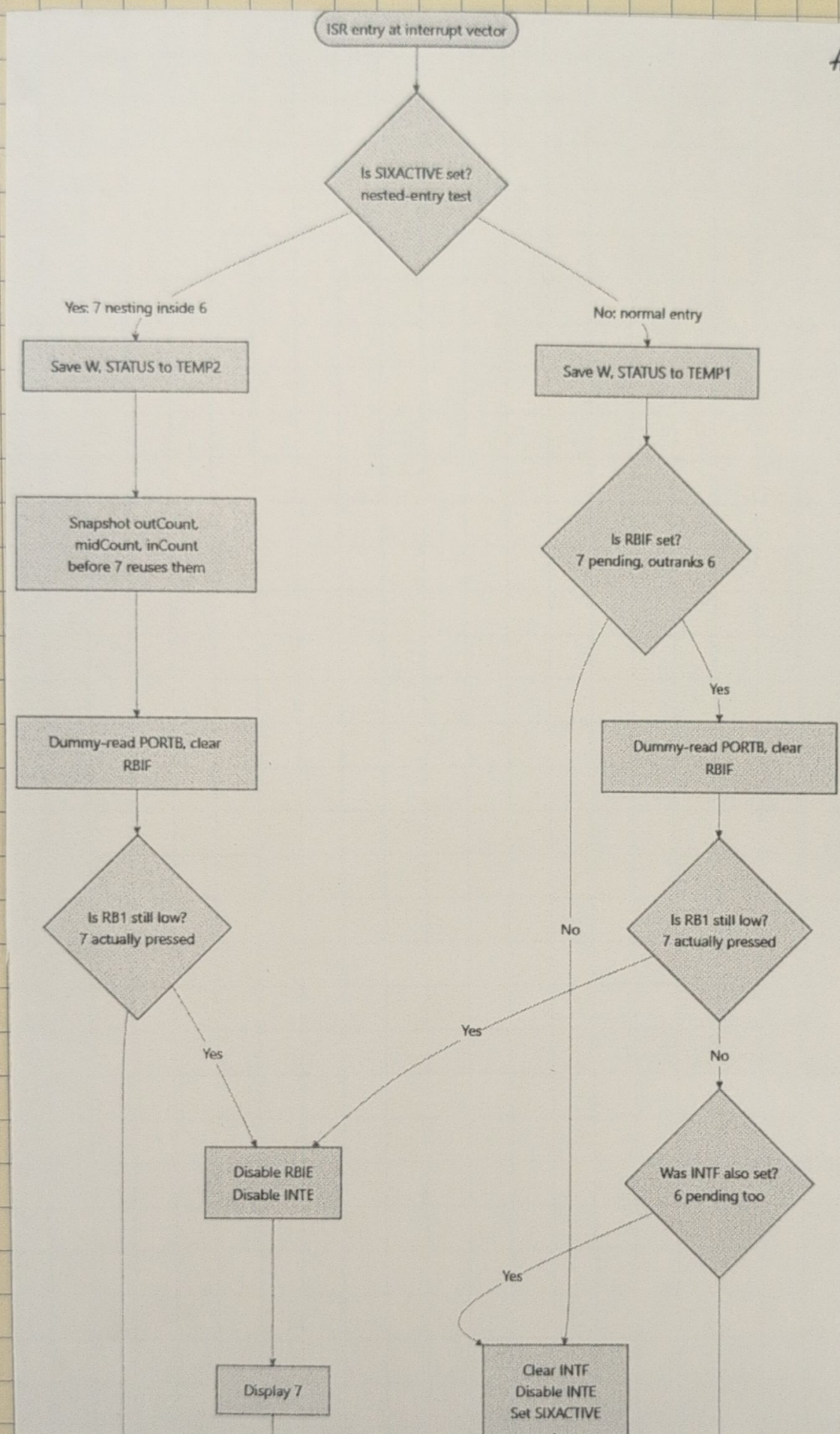


Figure 5.20 ISR Flowchart Part 1 continued on next pg

AND
9/25/26

AND 9/25/26

AND 9/25/26

Lab 5 Interrupt Service Routines

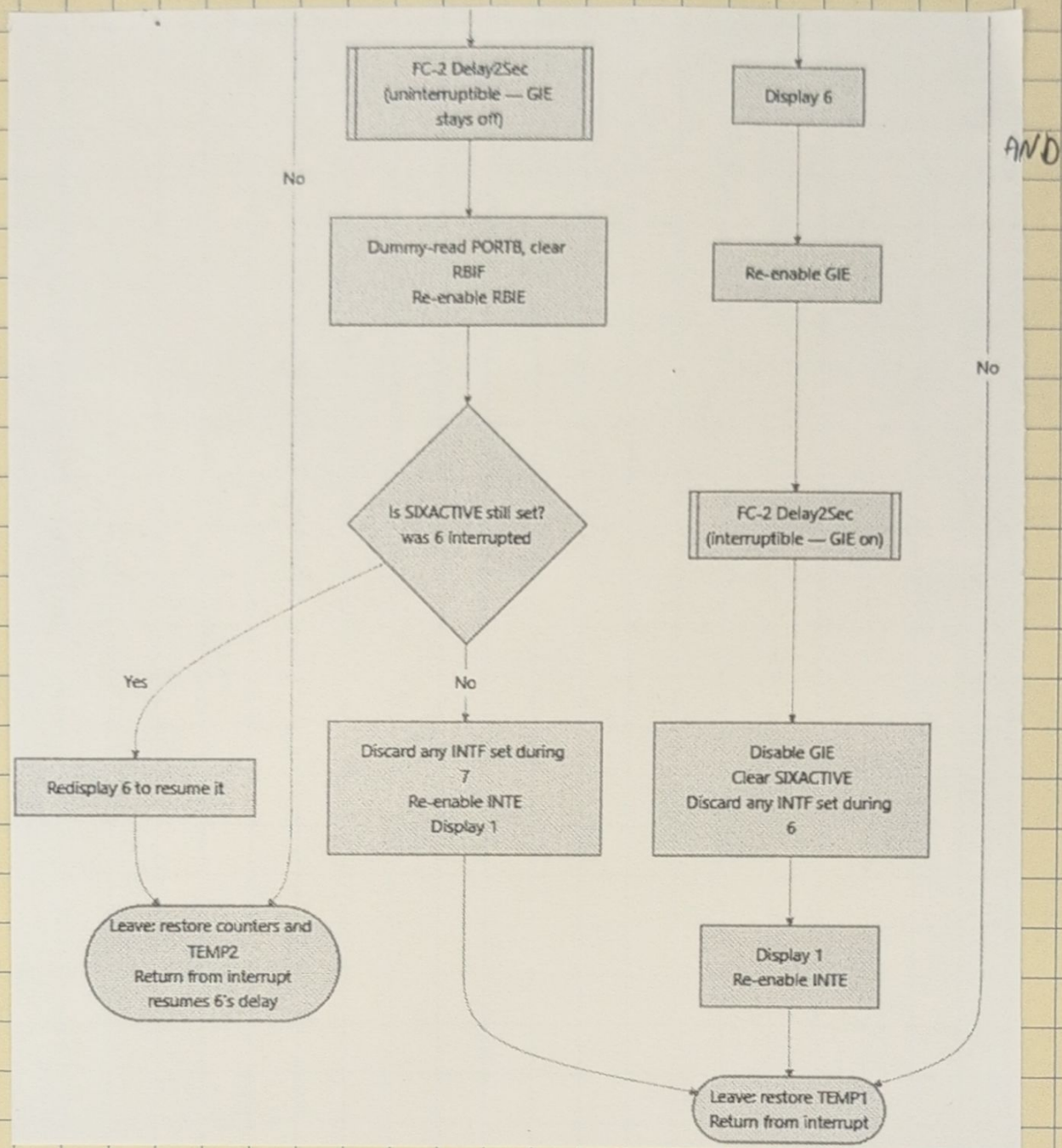
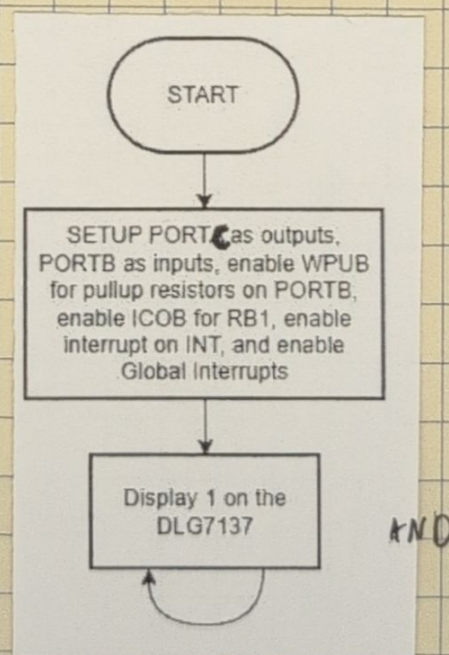
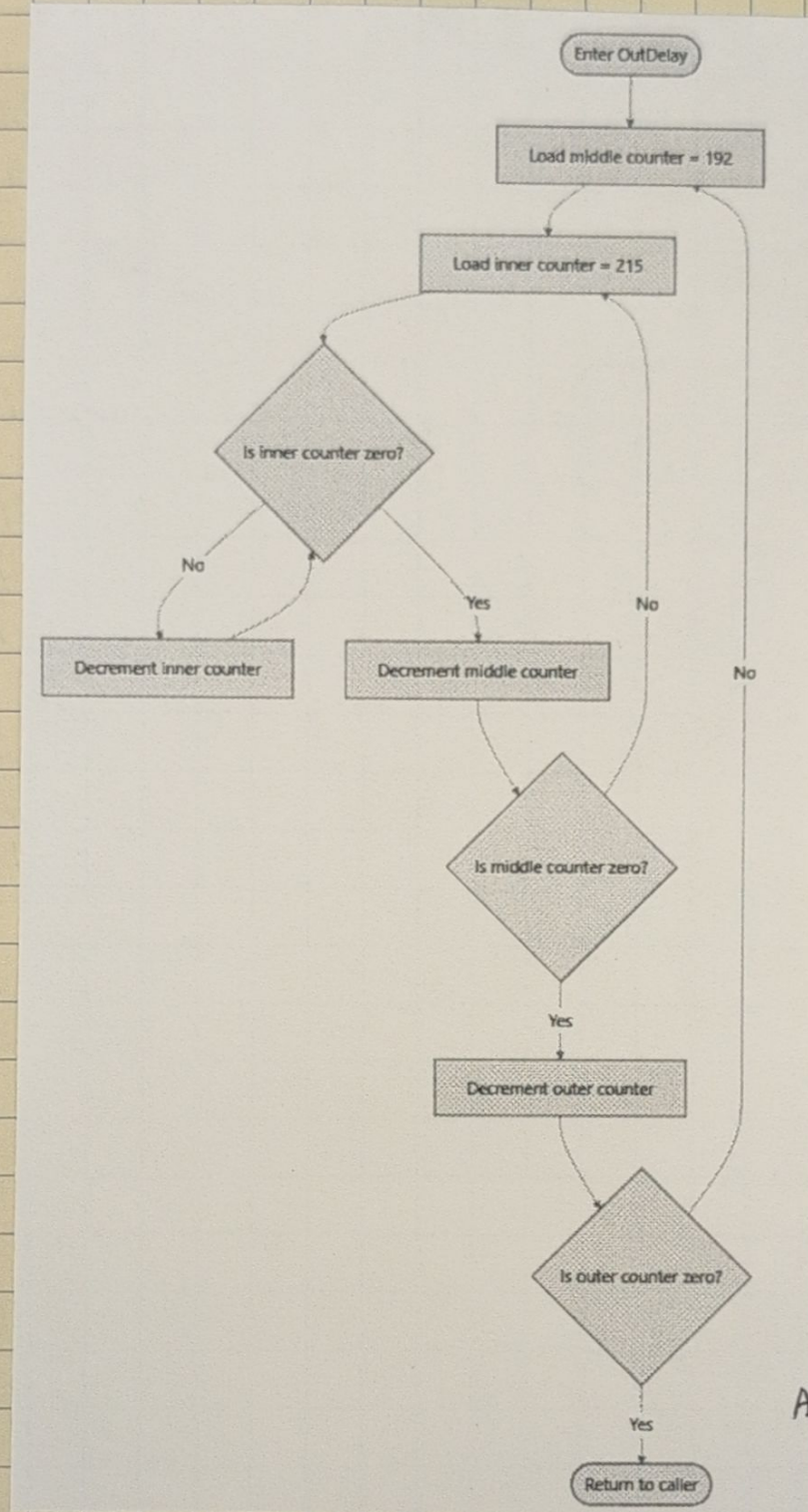
Figure S.22 ISR Flowchart Part 2 *see pt one on previous pg*

Figure S.23 Setup & Main Flowchart

AND 9/25/26



AND

Figure 5.24 Delay flowchart

AND
9/25/26

AND 9/25/26

AND 9/25/26

ISR ENTRY -- SIXACTIVE tells us, with a single bit test, whether this is a fresh entry from idle or a 7 nesting inside an already-running 6. While 6 is active, INTE is held off (see Service6), so a nested entry can only ever be caused by RBIF/RBIF -- i.e. it can only be a 7.

ISRHandler:

```
BTFSQ SIXACTIVE,0
goto SaveNested
goto SaveNormal
```

SaveNormal:

```
MOVWF W_TEMP1
SWAPF STATUS,W
MOVWF STATUS_TEMP1
BTFSQ INTCON,0 ; RBIF set? 2 outranks 6 -- check it first
goto CheckSeven_Normal
goto Service6
```

SaveNested:

```
MOVWF W_TEMP2
SWAPF STATUS,W
MOVWF STATUS_TEMP2
; 6's delay routine and 7's delay routine share counters -- protect
; 6's in-progress count before letting 7 reuse them
MOVWF outCount,W
MOVWF D2SCHT_SAVE
MOVWF midCount,W
MOVWF DCHT2_SAVE
MOVWF inCount,W
MOVWF DCHT1_SAVE
```

Both entry paths funnel here to tell a real RB1 press apart from a release (IOC fires on either edge).

CheckSeven_Normal:

```
MOVWF PORTB,W ; dummy read -- required to end the mismatch
BCF INTCON,0 ; clear RBIF
BTFSQ PORTB,1 ; still LOW (pressed)?
goto Service7
BTFSQ INTCON,1 ; release -- was a 6 also pending?
goto Service6
goto LeaveNormal
```

CheckSeven_Nested:

```
MOVWF PORTB,W
BCF INTCON,0
BTFSQ PORTB,1
goto Service7
goto LeaveNested ; release while nested -- just resume 6
```

AND

Checks if the interrupt that occurred was a normal or nested interrupt

Saves current W & Status registers before servicing the interrupt

Saves current W & Status registers for the interrupt 6 routine & saves the 3 delay counters so 7 doesn't override the progress of 6

Reads PORTB to clear the IOC mismatch, clears RBIF, & checks if RB1 is still low

Used to see when 7 interrupts 6 & checks if RB1 was an actual button press for the interrupt

Figure 5.25 Start of ISR Handler

SERVICE 6 (low priority). GIE is re-armed for the whole delay so 7 can preempt at any point. INTE stays off for the duration so a second RB0 edge can't re-trigger 6 on top of itself.

Service6:

```
BCF INTCON,1 ; clear INTF
BCF INTCON,4 ; INTE off -- ignore further RB0 edges while 6 runs
BSF SIXACTIVE,0 ; mark 6 active (enables nested-entry detection)
MOVWF D_SIX
MOVWF PORTC ; display 6
BSF INTCON,7 ; GIE back on -- 7 may now preempt this delay
CALL Delay2Sec ; interruptible 2-second delay

BCF INTCON,7 ; GIE off again while wrapping up (atomic finish)
BCF SIXACTIVE,0 ; 6 no longer active
BCF INTCON,1 ; discard any 6-request that arrived during 6 itself
MOVWF D_ONE
MOVWF PORTC ; 6 is done -- normal state is 1
BSF INTCON,4 ; INTE back on -- fully idle again
goto LeaveNormal
```

SERVICE 7 (absolute priority) -- ONE shared copy, used whether 7 was triggered from idle or nested inside 6. GIE is left OFF the entire time, so nothing (not even another 7) can preempt it.

Service7:

```
BCF INTCON,3 ; RBIE off -- nothing may preempt 7
BCF INTCON,4 ; INTE off -- nothing may preempt 7 (GIE already off)
MOVWF D_SEVEN
MOVWF PORTC ; display 7
CALL Delay2Sec ; UN-interruptible 2-second delay

MOVWF PORTB,W ; resync the IOC reference and discard anything
BCF INTCON,0 ; that happened to occur during 7
BSF INTCON,3 ; RBIE back on
```

```
BTFSQ SIXACTIVE,0
goto Service7_ResumeSix
```

```
BCF INTCON,1 ; discard a stray 6-request that arrived during 7
BSF INTCON,4 ; INTE back on -- fully idle
MOVWF D_ONE
MOVWF PORTC
goto LeaveNormal
```

Service7_ResumeSix:

```
MOVWF D_SIX
MOVWF PORTC ; 6 resumes -- redisplay 6 immediately
; (INTE intentionally stays off; 6 itself turns
; it back on when it finally completes)
goto LeaveNested
```

AND

Handles the RB0/INT interrupt for 6 & disables additional interrupts on RB0, sets SIXACTIVE to indicate 6 is active, displays 6 on DLG7137, re-enables GIE so that 7 can interrupt 6, runs a 2 sec interruptible delay, & then returns the display back to 1

Handles the RB1 interrupt for 7, disables both interrupt sources so that 7 can't be interrupted, displays 7, runs a 2 sec uninterruptible delay, clears and resynchronizes the IOC flag after the delay, & then if 6 was running before 7 it restores 6 or returns back to 1

Restarts the display of 6 after 7 leaves the 6 interrupt disabled until the original 6 completes, & then returns the saved 6

Figure 5.26 Service Routines in ISR Handler

AND 9/25/26


```

; CONTEXT RESTORE (datasheet SWAPF method)
LeaveNormal:
    SWAPF STATUS_TEMP1,W
    MOVWF STATUS
    SWAPF W_TEMP1,F
    SWAPF W_TEMP1,W
    RETFIE

LeaveNested:
    MOVF D2SCNT_SAVE,W
    MOVWF outCount
    MOVF DCNT2_SAVE,W
    MOVWF midCount
    MOVF DCNT1_SAVE,W
    MOVWF inCount
    SWAPF STATUS_TEMP2,W
    MOVWF STATUS
    SWAPF W_TEMP2,F
    SWAPF W_TEMP2,W
    RETFIE
    
```

Restores the saved W & STATUS registers, SWAPF method used to correctly restore the registers, & RETFIE returns to main program & re-enables interrupts

Restores the delay counters saved when 7 interrupted 6, restores the saved STATUS & W registers for the 6 routine, RETFIE returns to the interrupted 6 routine

Figure 5.27 Leave ISR routines

```

; DELAY -- ONE level of CALL nesting on purpose (Delay3Sec -> Delay500ms).
; Delay500ms itself uses GOTO-based nested loops, not CALL, so it never
; adds a second level. Keeping this flat is what keeps the worst-case
; hardware stack safely under the 8-level limit.
Delay2Sec:
    MOVW 0X04    ;1 TCY
    MOVWF outCount ;1 TCY
    CALL OutDelay
    MOVW 0X04    ;1 TCY
    MOVWF outCount ;1 TCY
    CALL OutDelay
    MOVW 0X04    ;1 TCY
    MOVWF outCount ;1 TCY
    CALL OutDelay
    MOVW 0X04    ;1 TCY
    MOVWF outCount ;1 TCY
    CALL OutDelay
    RETURN

OutDelay:
    MOVW 0XC0    ;1 TCY 192 in decimal
    MOVWF midCount ;1 TCY
    NOP
    NOP
    NOP
    NOP
    goto MidDelay ;2 TCY

MidDelay:
    MOVW 0XD7    ;1 TCY 215 in decimal
    MOVWF inCount ;1 TCY
    goto InDelay ;2 TCY

InDelay:
    DECFSE inCount ;1 (2) TCY
    goto InDelay ;2 TCY

    DECFSE midCount ;1 (2) TCY
    goto MidDelay ;2 TCY

    DECFSE outCount ;1 (2) TCY
    goto OutDelay ;2 TCY
    RETURN ;2 TCY
    
```

Figure 5.28 Delay routines

In the Lab:

1. Verify that main displays 1
2. Trigger RBX/INT & verify the approximately two-second 6 behavior followed by return to 1

Lab 5 Interrupt Service Routines

AND 9/25/26

3. Trigger RB1 IOC while 1 is active. Verify $1 \rightarrow 7 \rightarrow 1$
4. Trigger 7 while 6 is active. Verify $6 \rightarrow 7 \rightarrow 6 \rightarrow 1$, with 6 completing the time that remained when it was interrupted
5. Trigger 7 repeatedly while 6 is active at different points. Each 7 must complete without interruption, then return to the interrupted 6 until 6 eventually completes & returns to 1
6. Attempt to trigger 6 while 7 is active & verify that 6 doesn't interrupt 7
7. Compare observed execution with the stack depth analysis
8. The instructor will intentionally exercise the design near its worst-case nesting condition & may attempt to expose stack or context problems
9. Record & explain any ~~further~~ failure observed

TJR
9-25-26Conclusion:

This lab was about how interrupts work with the PIC16F883 and how to handle interrupt flags. In part 1 I used a button on RB0/INT and configured the interrupt to activate on a falling edge then increment a count for how many interrupts occurred on PORTC using LEDs. In part 2 using a similar set up to display the interrupt count on PORTC LEDs but use RB4 to increase the count & RB0/INT to clear the count. In part 3 every PORTB pin causes an interrupt but whichever pin RB7 $\rightarrow$ RB7 should of displayed its value on LEDs connected to PORTA, RA0 $\rightarrow$ RA2, but every change on PORTB caused an interrupt & because I used an 8-pin DIP switch every switch caused a change on the LED display. In part 4 I used nested interrupts to display a value on a PLG7137, 1 would always display until a button selected either 6 or 7 to display that value for 2 seconds on the display, however 7 was able to interrupt 6 so if 6 was on the display 7 took over for 2 sec then 6 returned to finish its time before the display returned to 1.

AND 9/25/26